

Text Deduplication

Exact Match

Question: Whether the two files are exactly the same?

Direct way: compare two files to see if each byte is the same.

Hash Method:

Check $\text{Hash}(\text{File A}) = \text{Hash}(\text{File B})$

The size of hash value is fixed and much smaller than the size of raw file.

Locality-Sensitive Hashing (LSH)

Definition:

A hashing technique where **similar items map to the same bucket** with high probability.

Goal:

Approximate nearest neighbor search **without brute-force comparisons**.

Traditional Hash vs. LSH:

- Traditional: **Slight** input change → **Completely** different hash (e.g., SHA-256).
- LSH: **Similar** inputs → **Similar/Matching** hashes.

Locality-Sensitive Hashing (LSH)

Implementations of LSH:

- MinHash
- SimHash
- Bit Sampling

Jaccard Similarity

Definition: A statistic to compare the similarity between two finite sets.

Formula:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

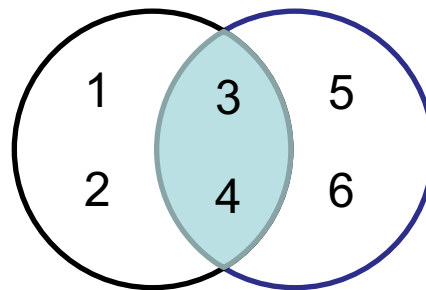
Ranges from 0 (no overlap) to 1 (identical sets).

Jaccard Similarity

Formula: $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$

Example:

- A: {1, 2, 3, 4}
- B: {3, 4, 5, 6}



$$|A \cap B| = |\{3, 4\}| = 2$$

$$|A \cup B| = |\{1, 2, 3, 4, 5, 6\}| = 6$$

$$J(A, B) = 2/6 = 0.33$$

Jaccard Similarity

Use Cases:

- Document Similarity
- Recommendation Systems
- Bioinformatics: Compare genetic or species datasets

Limitations:

- Ignore element frequency (only binary presence/absence)
- High computation overload

Min Hash

Goal: approximately compute Jaccard similarity in efficient way.

Key Idea:

Hash elements of the set, keep track of the minimum hash value for each set., and use the minimum hash values to estimate Jaccard similarity.

Min Hash

Steps:

- Define k different hash functions: $h_1, h_2, \dots, h_k$ that can randomly map input elements to integer values.
- Compute MinHash Signatures
 - For each hash function h_i , compute the hash values of all elements in sets A and B , then record the minimum hash value:

$$\text{MinHash}_i(A) = \min\{h_i(x) | x \in A\}$$

$$\text{MinHash}_i(B) = \min\{h_i(x) | x \in B\}$$

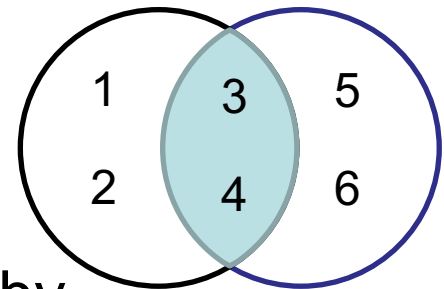
- This results in a signature of length k for each set.

Min Hash

Steps:

- Estimate Jaccard Similarity Using MinHash Collision Probability
 - The probability that $\text{MinHash}_i(A) = \text{MinHash}_i(B)$ for a randomly chosen hash function h_i is equal to the Jaccard similarity:

$$P(\text{MinHash}_i(A) = \text{MinHash}_i(B)) = J(A, B)$$



- Thus, we can estimate Jaccard similarity by computing the fraction of identical MinHash values across all k hash functions:

$$\hat{J}(A, B) = \frac{\text{Number of matching MinHash values}}{k}$$

Min Hash

Example: Suppose we have two sets of words (e.g., representing two documents):

- Set A = {apple, banana, mango, orange}
- Set B = {banana, mango, grape, peach}

Step 1:

1. Define two simple hash functions:

$$\text{Hash function 1: } h_1(x) = (2x + 1) \bmod 7$$

$$\text{Hash function 2: } h_2(x) = (3x + 4) \bmod 7$$

Min Hash

Example: Suppose we have two sets of words (e.g., representing two documents):

- Set A = {apple, banana, mango, orange}
- Set B = {banana, mango, grape, peach}

Step 2:

2. Compute the hash values for each element:

Element	ID	$h_1(x) = (2x + 1) \bmod 7$	$h_2(x) = (3x + 4) \bmod 7$
apple	1	$(2 \times 1 + 1) \bmod 7 = 3$	$(3 \times 1 + 4) \bmod 7 = 0$
banana	2	$(2 \times 2 + 1) \bmod 7 = 5$	$(3 \times 2 + 4) \bmod 7 = 3$
mango	3	$(2 \times 3 + 1) \bmod 7 = 0$	$(3 \times 3 + 4) \bmod 7 = 6$
orange	4	$(2 \times 4 + 1) \bmod 7 = 2$	$(3 \times 4 + 4) \bmod 7 = 2$
grape	5	$(2 \times 5 + 1) \bmod 7 = 4$	$(3 \times 5 + 4) \bmod 7 = 5$
peach	6	$(2 \times 6 + 1) \bmod 7 = 6$	$(3 \times 6 + 4) \bmod 7 = 1$

Min Hash

Example: Suppose we have two sets of words (e.g., representing two documents):

- Set A = {apple, banana, mango, orange}
- Set B = {banana, mango, grape, peach}

Step 3:

3. Compute MinHash Signatures:

MinHash Signature for Set A:

Set A contains {apple, banana, mango, orange}, so we extract the corresponding hash values:

h_1 values: {3, 5, 0, 2} → Min value = 0

h_2 values: {0, 3, 6, 2} → Min value = 0

Min Hash

Example: Suppose we have two sets of words (e.g., representing two documents):

- Set A = {apple, banana, mango, orange}
- Set B = {banana, mango, grape, peach}

Step 3:

3. Compute MinHash Signatures:

MinHash Signature for Set B:

Set A contains {banana, mango, grape, peach}, so we extract the corresponding hash values:

- h_1 values: {5, 0, 4, 6} → Min value = 0
- h_2 values: {3, 6, 5, 1} → Min value = 1

Min Hash

Example: Suppose we have two sets of words (e.g., representing two documents):

- Set A = {apple, banana, mango, orange}
- Set B = {banana, mango, grape, peach}

Step 4:

4. Compute estimated Jaccard Similarity:

$$\hat{J}(A, B) = \frac{\text{Number of matching hash values}}{\text{Total number of hash functions}}$$

MinHash signatures for A and B = [0, 0] and [0, 1]

Matching values: 1 out of 2

Estimated Jaccard Similarity = $1/2 = 0.5$

Min Hash

Example: Suppose we have two sets of words (e.g., representing two documents):

- Set A = {apple, **banana**, **mango**, orange}
- Set B = {**banana**, **mango**, grape, peach}

What is the true Jaccard similarity?

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{2}{6} = 0.333$$

Sim Hash

SimHash:

- Converts documents to fixed-size fingerprints
- Similar documents have similar fingerprints

Steps:

1. Feature Extraction & Weighting
 - Tokenize the text and extract keywords
 - Assign weights to each word
2. Hashing
 - Generate a standard hash (e.g., MD5 or SHA-1) for each keyword and convert it into a fixed-length binary vector.

Sim Hash

Steps:

1. Feature Extraction & Weighting
2. Hash
3. Weighted summation
 - Multiply each bit of the binary vector by the word's weight to obtain a weighted vector.
 - Sum all weighted vectors element-wise to produce a real-valued vector.
4. Generate fingerprints

Sim Hash

Steps:

1. Feature Extraction & Weighting
2. Hash
3. Weighted summation
4. Generate fingerprints
 - For each bit in the summed vector:
 - If the value ≥ 0 , set the bit to 1.
 - Else, set it to 0.
 - The resulting binary string is the SimHash fingerprint.

Sim Hash

Example:

For words “apple” (weight = 2) and “banana” (weight = 3), with hashes **1100** and **0110**

- Bit 1: $(2 \times 1) + (3 \times 0) = 2 \rightarrow 1$
- Bit 2: $(2 \times 1) + (3 \times 1) = 5 \rightarrow 1$
- Bit 3: $(2 \times 0) + (3 \times 1) = 3 \rightarrow 1$
- Bit 4: $(2 \times 0) + (3 \times 0) = 0 \rightarrow 0$

Final SimHash: **1110**.

Sim Hash

Exercise: Compute SimHash for 2 sentences:

- "I love discrete math"
- "I adore discrete maths"

Assume (weights are all 1):

Hash("I")=0001, Hash("discrete")=1010,

Hash("love")=0101, Hash("adore")=0100,

Hash("math")=1100, Hash("maths")=1101

Sim Hash

Exercise: Compute SimHash for 2 sentences:

- "I love discrete math"
- "I adore discrete maths"

Assume (weights are all 1):

Hash("I")=0001, Hash("discrete")=1010,

Hash("love")=0101, Hash("adore")=0100,

Hash("math")=1100, Hash("maths")=1101

Solution:

Sentence 1: $0001+0101+1010+1100=2212=1111$

Sentence 1: $0001+0100+1010+1101=1212=1111$

Bit Sampling Hash

Goal:

- Rapidly filter dissimilar data using a single random bit.

Hash Function Construction:

- Randomly select one bit position (uniformly from 1 to d).
- Hash value = value of the vector at the selected position.

Example:

Vector $u = [1, 0, \mathbf{1}, 1, 0]$, selected bit = 3 $\rightarrow$ $\text{hash}(u) = 1$

Vector $v = [0, 0, \mathbf{1}, 1, 0]$, selected bit = 3 $\rightarrow$ $\text{hash}(v) = 1$

Result: Same hashes $\Rightarrow$ Likely similar

Bit Sampling Hash

Example (sample 2 bits):

Vector $u = [1, 0, 1, 1, 0]$, selected bit = 1, 3 $\rightarrow$ hash(u) = [1, 1]

Vector $v = [0, 0, 1, 1, 0]$, selected bit = 1, 3 $\rightarrow$ hash(v) = [0, 1]

Result: Hashes differ $\Rightarrow$ Likely dissimilar

Similarity metric: Hamming distance (# of differing bits).

Optimization: Sampling strategy.